

EXP 6 – VERSION 2

FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS

AIM

To implement Feedforward Neural Network (FNN) and Convolutional Neural Network (CNN) models using TensorFlow and Keras for image classification using the **Fashion-MNIST and MNIST datasets**.

PROCEDURE

1. Import the required Python libraries.
2. Load the Fashion-MNIST dataset for the FNN.
3. Normalize and flatten the input images.
4. Build and compile an FNN with different dense layers.
5. Train and evaluate the FNN model.
6. Load the MNIST dataset for the CNN.
7. Normalize and reshape the images.
8. Build and compile a CNN with convolution, pooling and dropout layers.
9. Train and evaluate the CNN model.
10. Display sample predictions and accuracy.

CODE

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.datasets import fashion_mnist, mnist

(x_train_f, y_train_f), (x_test_f, y_test_f) = fashion_mnist.load_data()

x_train_f = x_train_f.astype("float32") / 255.0
x_test_f = x_test_f.astype("float32") / 255.0

x_train_f = x_train_f.reshape(-1, 784)
x_test_f = x_test_f.reshape(-1, 784)

fnn = Sequential([
    Input(shape=(784,)),
    Dense(256, activation="relu"),
    Dropout(0.3),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

fnn.compile(
    optimizer="adam",
```

```

    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history_fnn = fnn.fit(
    x_train_f,
    y_train_f,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

loss_fnn, accuracy_fnn = fnn.evaluate(
    x_test_f,
    y_test_f,
    verbose=0
)

print("FNN Test Loss:", loss_fnn)
print("FNN Test Accuracy:", accuracy_fnn)

fnn_prediction = fnn.predict(x_test_ff[:10], verbose=0)
fnn_predicted = np.argmax(fnn_prediction, axis=1)

print("Actual Labels:", y_test_ff[:10])
print("Predicted Labels:", fnn_predicted)

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test_ff[i].reshape(28, 28), cmap="gray")
    plt.title("Actual: " + str(y_test_ff[i]) + "\nPred: " + str(fnn_predicted[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()

(x_train_c, y_train_c), (x_test_c, y_test_c) = mnist.load_data()

x_train_c = x_train_c.astype("float32") / 255.0
x_test_c = x_test_c.astype("float32") / 255.0

x_train_c = x_train_c.reshape(-1, 28, 28, 1)
x_test_c = x_test_c.reshape(-1, 28, 28, 1)

cnn = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (5, 5), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation="relu"),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.4),
    Dense(10, activation="softmax")
])

cnn.compile(

```

```

optimizer="adam",
loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)

history_cnn = cnn.fit(
    x_train_c,
    y_train_c,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

loss_cnn, accuracy_cnn = cnn.evaluate(
    x_test_c,
    y_test_c,
    verbose=0
)

print("CNN Test Loss:", loss_cnn)
print("CNN Test Accuracy:", accuracy_cnn)

prediction = cnn.predict(x_test_c[:10], verbose=0)
predicted = np.argmax(prediction, axis=1)

print("Actual Labels:", y_test_c[:10])
print("Predicted Labels:", predicted)

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test_c[i].reshape(28, 28), cmap="gray")
    plt.title("Actual: " + str(y_test_c[i]) + "\nPred: " + str(predicted[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(history_fnn.history["accuracy"], label="FNN Accuracy")
plt.plot(history_fnn.history["val_accuracy"], label="FNN Validation Accuracy")
plt.plot(history_cnn.history["accuracy"], label="CNN Accuracy")
plt.plot(history_cnn.history["val_accuracy"], label="CNN Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("FNN and CNN Accuracy")
plt.legend()
plt.show()

```

OUTPUT

```

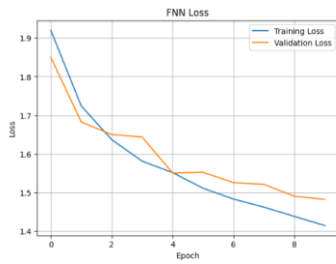
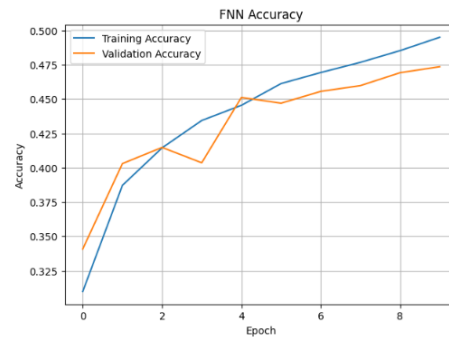
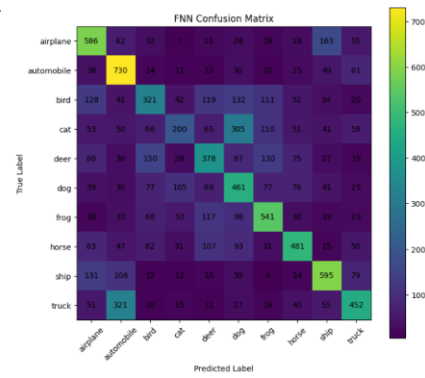
* FNN Test Loss: 1.4822
FNN Test Accuracy: 0.4745

FNN Classification Report
precision    recall  f1-score   support

airplane    0.49    0.59    0.53    1000
automobile  0.58    0.73    0.66    1000
bird        0.48    0.32    0.39    1000
cat         0.40    0.20    0.27    1000
deer        0.41    0.28    0.33    1000
dog         0.36    0.46    0.40    1000
frog        0.52    0.54    0.53    1000
horse       0.55    0.48    0.51    1000
ship        0.57    0.59    0.58    1000
truck       0.53    0.45    0.49    1000

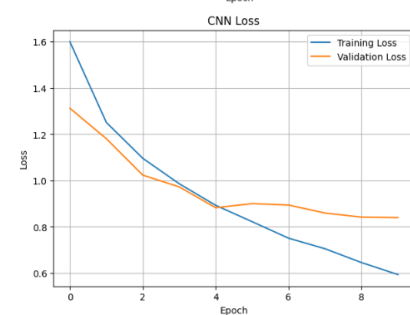
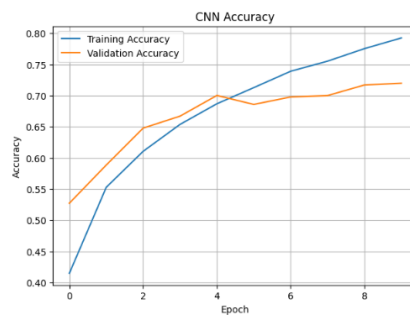
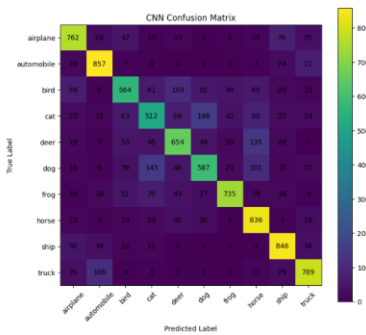
accuracy    0.47    0.47    0.47    10000
macro avg   0.47    0.47    0.47    10000
weighted avg 0.47    0.47    0.47    10000

```



CNN Test Loss: 0.8688
CNN Test Accuracy: 0.7142

CNN Classification Report	precision	recall	f1-score	support
airplane	0.75	0.76	0.76	1000
automobile	0.78	0.86	0.82	1000
bird	0.66	0.56	0.61	1000
cat	0.58	0.51	0.54	1000
deer	0.67	0.65	0.66	1000
dog	0.65	0.59	0.62	1000
frog	0.83	0.73	0.78	1000
horse	0.66	0.84	0.74	1000
ship	0.78	0.85	0.81	1000
truck	0.77	0.79	0.78	1000
accuracy			0.71	10000
macro avg	0.71	0.71	0.71	10000
weighted avg	0.71	0.71	0.71	10000



RESULT

Thus, the **Feedforward Neural Network (FNN)** and **Convolutional Neural Network (CNN)** were successfully implemented using the **Fashion-MNIST** and **MNIST** datasets. The FNN classified flattened Fashion-MNIST images using fully connected layers, while the CNN classified MNIST images using convolution and pooling layers. The CNN effectively learned spatial features from images and provided good classification performance.